[ACCOUNT LAYER WEEK / DAY 03]

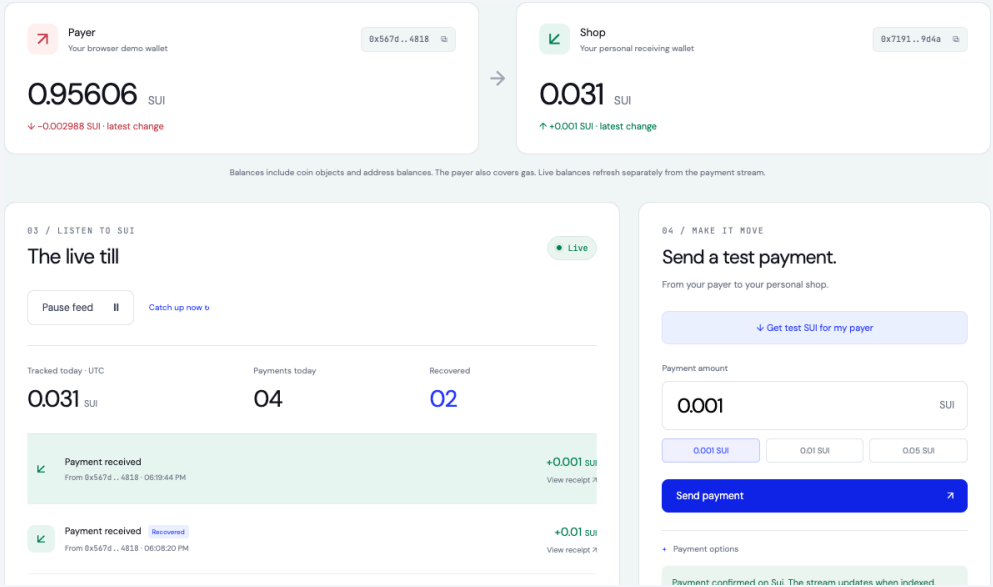
Real-time Sui.

A payment happens.
Your app should know.

GRAPHQL SUBSCRIPTIONS / THE NEW DATA STACK

SUIHUB LAGOS / ACCOUNT LAYER WEEK

Meet your live till.



PERSONAL

Your payer. Your shop.

LIVE

A receipt from the stream.

RECOVERABLE

A saved place to resume.

Live app: day3-till.vercel.app / Real testnet transactions. Personal student wallets.

Four layers. Four different jobs.

01 / gRPC

Read and act now.

Balances, objects, execution and checkpoints.

02 / GRAPHQL

Ask for a screen.

Filtered history, selected fields and subscriptions.

03 / ARCHIVE

Keep the past.

History beyond a full node’s retention.

04 / YOUR INDEXER

Own the data model.

Durable aggregates, joins and reprocessing.

One document. Any shop.

```
subscription Till($shop: SuiAddress!, $after: String, $afterCheckpoint: UInt53) {
 transactions(
   filter: { affectedAddress: $shop, afterCheckpoint: $afterCheckpoint }, after: $after
 ) {
   cursor
   node {
     digest sender { address }
     effects {
       status timestamp checkpoint { sequenceNumber }
       balanceChanges(first: 50) { nodes { owner { address } coinType { repr } amount } }
     }
   }
 }
}
```

Select a small payload. Every matching transaction carries a cursor.

Choose a filter. Open the stream.

TRANSACTIONS

Who or what?

affectedAddress · affectedObject · sentAddress

EVENTS

Which event?

sender · module · type + checkpoint bounds.

CHECKPOINTS

Follow chain progress.

Ordered delivery with a saved resume position.

SSE OVER POST

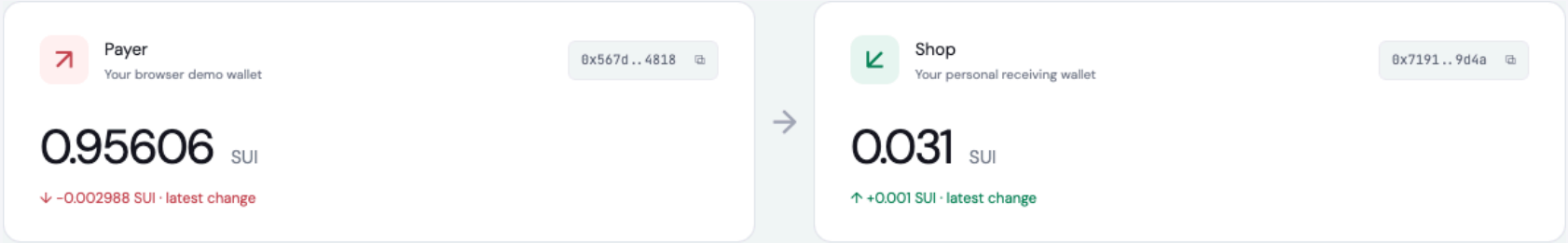
Use fetch to listen.

EventSource cannot send the GraphQL query body.

Watch the money move.

PAYER / MONEY OUT

SHOP / MONEY IN

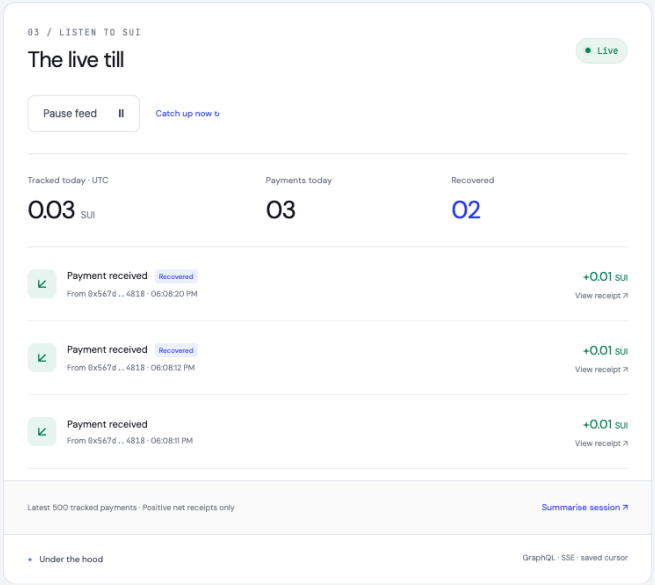


Payment + gas

Payment received

Payer ↓ payment + gas / Shop ↑ received value / Balances refresh independently of the feed

It catches up from where you stopped.



01 / PAUSE

The listener stops.

02 / PAY

The chain keeps going.

03 / RESUME

Missed receipts return.

Saved position → historical catch-up → live delivery on the same connection

No cursor yet? Keep a checkpoint.

01

First open

Capture an opening checkpoint

02

First delivery

Save the cursor with its payment

03

Reconnect

Use after, or the opening checkpoint

04

Continue

Replay first. Then follow live.

The cursor belongs in the same commit.

```
db.exec('BEGIN');
try {
 for (const p of payments) {
   insert.run(p.digest, p.coinType, p.amount,
     p.sender, p.checkpoint, p.timestamp, p.cursor);
 }
 saveCursor.run(shop, d.transactions.cursor,
   new Date().toISOString());
 db.exec('COMMIT');
} catch (e) {
 db.exec('ROLLBACK');
 throw e;
}
```

pnpm indexer → pnpm totals / PRIMARY KEY (digest, coin_type) / One commit

A full node can forget. History remains.



Payment record

SUCCESS

Digest	ArCwXsZUTywBuM4GbNX4CVMERmBBgeYY2mgmJJjdHa3a ↗
Recorded	Tue, 07 Oct 2025 22:04:44 GMT
Checkpoint	250,000,000

Balance changes

0xcdf2..4eb8	-0.006844038 SUI
0xcdf2..4eb8	+0.00000017 nBTC

Source: Sui GraphQL. A sender's net change can include gas.

DATE

7 October 2025

CHECKPOINT

250,000,000

RESULT

GraphQL found the receipt.

Verified 15 October 2026 / Transaction dated 7 October 2025 / Checkpoint 250,000,000

Plan for limits and real-world gaps.

PAGINATION

20 default / 50 max

Observed for Query.transactions. Page longer history.

QUERY TIMEOUT

40 seconds

Keep selections small. Provider settings can change.

LIVE FILTER GAP

Confirmed, but missing?

Address-balance deposits appeared after catch-up.

PRODUCTION

Check your provider.

Throttling, retention, quotas, auth and uptime matter.

Migrate the intention, not just the method.

```
pnpm migrate:legacy   # inspect the JSON-RPC error body
pnpm migrate:grpc     # current balance through gRPC
pnpm migrate:graphql  # structured indexed query

// gRPC SDK shape used by the app:
const { balance } = await client.getBalance({
 owner: address, coinType: '0x2::sui::SUI'
});
// balance.balance includes coin and address balances.
```

Execute and read current state with gRPC. Query indexed history with GraphQL.

The lifecycle is part of the demo.

REFRESH

Resume explicitly.

Open till restores the saved per-shop position.

SWITCH ACCOUNT

Switch the shop too.

A connected payer gets its own personal receiving address.

RESET SESSION

Clear tracking only.

Existing onchain funds and wallet keys stay in place.

UPGRADE

Treat cursors carefully.

Version local state. Rebuild if query semantics change.

Your turn: prove all four.

01

Isolation

Two browsers show different shops

02

Payment

A receipt matches a real digest

03

Recovery

Pause, pay, resume and reload

04

History

Find the October 2025 receipt

day3-till.vercel.app / Pay, pause, recover, reload. Then inspect an old receipt.

Make it live.
Make it recover.

day3-till.vercel.app

github.com/Nuel-osas/day3-till

YOUR NEXT BUILD / A LIVE FEED WITH A MEMORY